

Improving Sampling from Boltzmann Distributions using Diffusive Reverse KL

Luran Wang

Jan 2025

Abstract

Training generative models to sample from unnormalized density functions is a critical challenge in machine learning. Traditional methods utilize MCMC but suffer from slow sampling speed. An alternative is to train a neural sampler by minimizing the reverse Kullback-Leibler (KL) divergence due to its tractability; however, this approach leads to mode-seeking behavior, limiting the accurate approximation of multi-modal distributions. To address this, diffusive reverse KL divergence (DiKL) has been introduced, using Gaussian convolution to reduce the effective number of modes. We extend this by introducing multi-step DiKL, which minimizes the reverse KL along diffusion trajectories of both model and target densities, increasing model capacity. We validate our method on MOG experiments and propose integrating post-processing Langevin Dynamics as future work.

1 Introduction

Sampling from unnormalized distributions is a crucial research problem with applications in machine learning, Bayesian inference, and statistical mechanics. Consider a target distribution with an analytically defined but unnormalized density function:

$$p_d(x) = \exp(-E(x))/Z, \quad (1)$$

where $E(x)$ represents a differentiable energy function and Z is the intractable normalization constant.

A common approach to sampling from involves designing Markov Chain Monte Carlo (MCMC) samplers. However, for high-dimensional, multi-modal target distributions, MCMC methods converge slowly and require long chains for sample independence [6]. These limitations motivate alternative solutions such as neural samplers [3], which approximate the target distribution with a generative model $p_\theta(x)$. Training a neural sampler requires minimizing a divergence measure between $p_\theta(x)$ and target distribution $p_d(x)$.

Given the unavailability of samples of target distribution, the reverse KL divergence is used:

$$KL(p_\theta|p_d) = \mathbb{E}_{p_\theta} \left[\log \frac{p_\theta(x)}{p_d(x)} \right] \quad (2)$$

which exhibits mode-seeking behavior [1], leading to inadequate coverage of the full distribution. This issue is particularly problematic when contains multiple modes, as will underestimate variance and collapse into a few dominant regions.

To address this issue, we propose an alternative objective, the *diffusive KL divergence* (DiKL) [4], which convolves both the target and model distributions with Gaussian diffusion kernels, promoting mode connectivity and reducing mode collapse. This objective still defines a valid divergence but provides better mode coverage capabilities.

2 Methodology

2.1 Diffusive Reverse KL

Gaussian convolution is an effective method for bridging and merging isolated modes, widely used in diffusion models. It can also reduce the number of modes by convexifying functions through Gaussian smoothing.

Inspired by it, the diffusive KL (DiKL) divergence is defined as:

$$\text{DiKL}(p||q) = \sum_{t=1}^T w(t) \text{KL}(p * k_t || q * k_t), \quad (3)$$

where $\{k_t\}$ is a set of Gaussian kernels and $*$ denotes convolution. This technique promotes gradual connectivity between modes.

[8] shows minimizing DiKL ensures that the neural sampler approximates the target distribution:

$$D(\tilde{p}||q) = 0 \Leftrightarrow D(\tilde{p}||\tilde{q}) = 0, \quad (4)$$

$$\Leftrightarrow \tilde{p} = \tilde{q} \quad (5)$$

$$\Leftrightarrow \mathcal{F}\{k\}\mathcal{F}\{p\} = \mathcal{F}\{k\}\mathcal{F}\{q\}, \quad (6)$$

$$\Leftrightarrow \mathcal{F}\{p\} = \mathcal{F}\{q\}, \quad (7)$$

$$\Leftrightarrow p = q \Leftrightarrow \mathbb{P} = \mathbb{Q}. \quad (8)$$

2.2 Training a Neural Sampler with DiKL

The gradient of the DiKL loss is:

$$\nabla_{\theta} \text{DiKL}_{k_t}(p_{\theta}||p_d) = \nabla_{\theta} \text{KL}(p_{\theta} * k_t || p_d * k_t), \quad (9)$$

which expands as:

$$\int p_{\theta}(x_t) (\nabla_{x_t} \log p_{\theta}(x_t) - \nabla_{x_t} \log p_d(x_t)) \frac{\partial x_t}{\partial \theta} dx_t. \quad (10)$$

A score model $s_{\phi}(x_t)$ is trained using denoising score matching [7]:

$$\int \|s_{\phi}(x_t) - \nabla_{x_t} \log k(x_t|x)p_{\theta}(x|x_t)\|_2^2 p_{\theta}(x_t) dx_t, \quad (11)$$

$$= \int \int \|s_{\phi}(x_t) - \nabla_{x_t} \log k(x_t|x)\|_2^2 p_{\theta}(x, x_t) dx dx_t + \text{const}. \quad (12)$$

To analyze the target score function, a linear combination of denoising score identity and target score identity (TSI) [2] is used:

$$\nabla_{x_t} \log p_d(x_t) = \int (\alpha_t(x + \nabla_x \log p_d(x)) - x_t) p_d(x|x_t) dx. \quad (13)$$

To approximate the posterior $p_d(x_t|x)$, Hamiltonian Monte Carlo (HMC) is employed, yielding:

$$\nabla_x \log p_d(x_t) = -\nabla_x E(x) - \frac{\alpha_t(\alpha_t x - x_t)}{\sigma_t^2}. \quad (14)$$

This enables the neural sampler to efficiently approximate the target distribution, ensuring robust mode coverage.

2.3 Multi-step Diffusive KL Divergence

One limitation of the sampler trained using the diffusive KL (DiKL) divergence is its restricted capacity when used as a one-step sampler. While this approach generates high-quality samples, its ability to capture complex distributions remains constrained. To improve performance, a multi-step denoising strategy can be adopted, inspired by diffusion models.

2.3.1 Motivation for Multi-step Sampling

Similar to diffusion models, the process begins with the introduction of Gaussian noise through convolution with Gaussian kernels. Specifically, we define the kernel as:

$$k_t(\tilde{x}|x) = \mathcal{N}(\sqrt{1 - \beta_t^2}x, \beta_t I), \quad (15)$$

where β_t represents the noise variance at time step t . This gradually transforms the target distribution into a Gaussian-like distribution, as illustrated in Figure 1.

Unlike traditional diffusion models, in our case, the target distribution x_T is sampled directly, and the true values of intermediate states x_t are unavailable. This necessitates a modification in how the denoising process is handled.

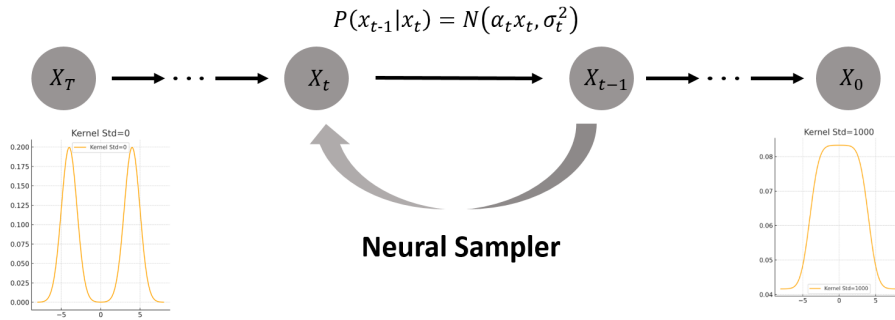


Figure 1: Visualization of the multi-step denoising process using DiKL.

2.3.2 Loss Function and Multi-step Sampling

To formalize this approach, we optimize the following objective:

$$\theta = \arg \min_{\theta} \text{DiKL} \left(f_t^{\theta} \left(\tilde{f}_{t-1}(x) \right) * k_t \middle| \middle| p_d * \tilde{k}_t \right), \quad (16)$$

where $\tilde{k}_t$ represents the cumulative application of Gaussian convolution kernels:

$$\tilde{k}_t = k_T \circ k_{T-1} \circ \dots \circ k_{t+1}. \quad (17)$$

This formulation allows the neural sampler to iteratively refine noisy samples, improving their quality over multiple steps.

Figure 2 illustrates this iterative denoising process. At each step, the function f_t removes a portion of the noise introduced by the forward process. The full sequence of transformations is expressed as:

$$\tilde{f}_{t-1} = f_{t-1} \circ f_{t-2} \circ \dots \circ f_1. \quad (18)$$

The process starts from an initial noisy sample x_T and applies successive transformations to recover the clean sample x_0 .

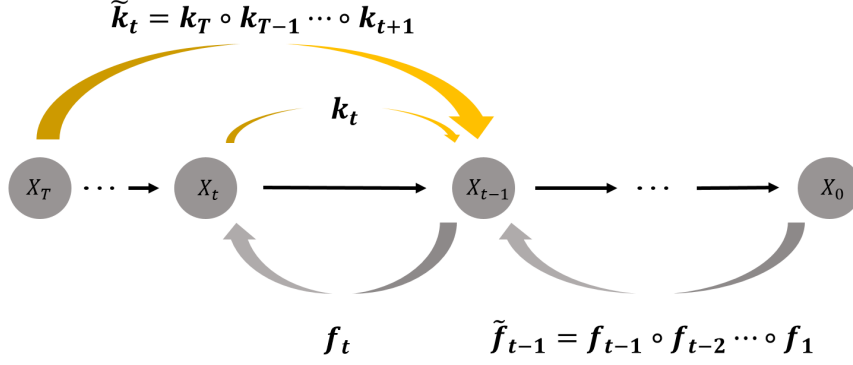


Figure 2: Illustration of the multi-step DiKL process, where the learned transformations progressively refine noisy samples.

2.3.3 Gradient Computation for Efficient Training

A primary challenge in multi-step sampling is the increased computational cost. A naive implementation would require an independent training step for each iteration, leading to prohibitive overhead. To mitigate this, we derive an efficient formulation for the gradient of the DiKL loss at each step:

$$\nabla_{\theta} D_{KL}(p_{\theta} || p_d) = \int \left(\frac{\alpha E_{p(x|x_t)}[x] - x_t}{\beta^2} - \nabla_{x_t} \log p_d(x_t) \right) \frac{\partial x_t}{\partial \theta} p_{\theta}(x_t) dx_t. \quad (19)$$

This formulation enables training without explicitly learning a separate model for $\nabla_{x_t} \log p_{\theta}(x_t)$, reducing computational complexity. The term $E_{p(x|x_t)}[x]$ can be efficiently estimated using Hamiltonian Monte Carlo (HMC).

3 Results

To assess the effectiveness of the proposed multi-step diffusive reverse KL sampling approach, we conducted experiments on the MOG-3 dataset. This dataset consists of a three-component 2D Gaussian mixture with a location scaling of 15 and a log-variance scaling of 0.01, which serves as the target distribution. To validate our method, we employ a two-step sampling procedure, ensuring that the hyperparameters remain consistent with those used in the single-step neural sampler.

In particular, an additional intermediate time step $t = 0.5$ is introduced between time steps 1 (representing the data distribution) and 0 (denoting noise). The Gaussian kernels are defined as:

$$\tilde{k}_{0.5}(\tilde{x}|x) = \mathcal{N}(0.5x, \sqrt{0.75}), \quad k_{0.5}(x|x) = \mathcal{N}(0.2x, 0.82I), \quad (20)$$

$$\tilde{k}_{0.1}(\tilde{x}|x) = \mathcal{N}(0.1x, \sqrt{0.99}I). \quad (21)$$

The training of the two-step model is performed using the following optimization steps:

$$\theta_1 = \arg \min_{\theta_1} \text{DiKL} \left(f_{t_1}^{\theta_1}(x) * k_{0.5} || p_d * \tilde{k}_{0.1} \right), \quad (22)$$

$$\theta_2 = \arg \min_{\theta_2} \text{DiKL} \left(f_{t_2}^{\theta_2}(f_{t_1}^{\theta_1}(x)) * \tilde{k}_{0.5} || p_d * \tilde{k}_{0.5} \right). \quad (23)$$

The results, as illustrated in Figure 3, indicate that our method significantly mitigates the issue of mode-seeking.

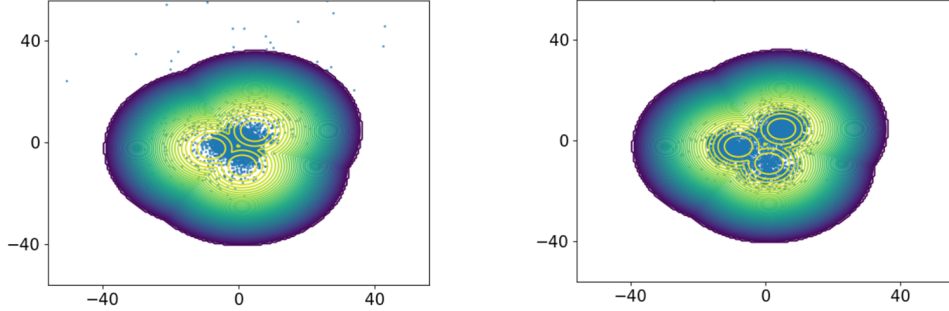


Figure 3: Results of the two-step sampling process.

4 Future Work

An alternative method to increase the model capacity is to add the post-processing Langevin dynamics to get final distribution $\tilde{p}_{\theta,\lambda}(x)$. While the neural sampler trained using the diffusive KL (DiKL) divergence achieves high-quality samples, its ability to fully capture complex distributions remains constrained. Future work can integrate Bayesian learning [5] to dynamically adapt these hyperparameters, improving convergence and efficiency.

To optimize the hyperparameters of Langevin updates, we introduce a Bayesian framework where λ is treated as a random variable with a prior distribution $p(\lambda)$. The posterior is given by:

$$p(\lambda|\text{data}) \propto p(\text{data}|\lambda)p(\lambda), \quad (24)$$

where $p(\text{data}|\lambda)$ measures the likelihood of the sampled distribution approximating the target distribution under DiKL. The prior can be chosen as:

$$p(\alpha) = \log \mathcal{N}(\mu_\alpha, \sigma_\alpha^2), \quad p(L) = \text{Discrete}(1, 10). \quad (25)$$

This ensures that α remains positive and is within a reasonable range.

Since directly computing the posterior is intractable, we introduce a variational distribution $q_\phi(\lambda)$, optimized by minimizing the KL divergence:

$$\mathbb{E}_{\lambda \sim q_\phi(\lambda)} [\log p(\text{data}|\lambda)] - D_{\text{KL}}(q_\phi(\lambda) \| p(\lambda)). \quad (26)$$

where considering the unavailability of samples from data distribution, we can set:

$$p(\text{data} | \lambda) \propto \exp(-D(\tilde{p}_{\theta,\lambda} \| p_0)), \quad (27)$$

Meanwhile the neural sampler could be trained by:

$$\theta = \arg \min_{\theta} \mathbb{E}_{\lambda \sim q_\phi(\lambda)} [\text{DiKL}(p_\theta * k_t \| p_d * k_t)]. \quad (28)$$

This allows the sampler to dynamically refine its hyperparameters during training, enhancing both efficiency and performance.

References

- [1] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [2] Valentin De Bortoli, Michael Hutchinson, Peter Wirsberger, and Arnaud Doucet. Target score matching. *arXiv preprint arXiv:2402.08667*, 2024.
- [3] Lauro Langosco di Langosco, Vincent Fortuin, and Heiko Strathmann. Neural variational gradient descent. *arXiv preprint arXiv:2107.10731*, 2021.
- [4] Jiajun He, Wenlin Chen, Mingtian Zhang, David Barber, and José Miguel Hernández-Lobato. Training neural samplers with reverse diffusive kl divergence. *arXiv preprint arXiv:2410.12456*, 2024.
- [5] Radford M Neal. *Bayesian learning for neural networks*, volume 118. Springer Science & Business Media, 2012.
- [6] Emilia Pompe, Chris Holmes, and Krzysztof Łatuszyński. A framework for adaptive mcmc targeting multimodal distributions. 2020.
- [7] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural computation*, 23(7):1661–1674, 2011.
- [8] Mingtian Zhang, Peter Hayes, Thomas Bird, Raza Habib, and David Barber. Spread divergence. In *International Conference on Machine Learning*, pages 11106–11116. PMLR, 2020.